SECUREGUARD ENTERPRISE

Behavior-Based Endpoint Detection & User Behaviour Analytics (UBA) System for Insider Threat Identification

A MINI PROJECT REPORT

Submitted by

SHAFAAT MUHAMMUD S
(922524118049)

in partial fulfillment for the award of the degree
of
BACHELOR OF ENGINEERING
in
COMPUTER AND COMMUNICATION ENGINEERING

V.S.B. ENGINEERING COLLEGE, KARUR – 639111
(AN AUTONOMOUS INSTITUTION)
ANNA UNIVERSITY :: CHENNAI 600 025

JULY 2026

BONAFIDE CERTIFICATE

Certified that this mini project report titled "SECUREGUARD ENTERPRISE – BEHAVIOR-BASED ENDPOINT DETECTION & USER BEHAVIOUR ANALYTICS (UBA) SYSTEM FOR INSIDER THREAT IDENTIFICATION" is the bonafide work of "SHAFAAT MUHAMMUD S (922524118049)" who carried out the project work under my supervision.

SIGNATURE

Mr. R. R. JEGAN, M.E., (Ph.D.,)

HEAD OF THE DEPARTMENT

Department. of CCE

V.S.B. Engineering College,

Karur-639111

SIGNATURE

Mr. BASKAR, M.E., (Ph.D.)

PROJECT MENTOR

Assistant professor

Department. of CCE

V.S.B. Engineering College,

Karur-639111

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

First and foremost, we express our thanks to our parents for providing us with a very nice environment for doing this project. We wish to express our sincere thanks to our founder and Chairman **Shri. V. S. BALSAMY, B.Sc., L.L.B.,** for his endeavour in educating us in this premier institution.

.

We extend our gratitude to **Dr. C. VENNILA, M.E., Ph.D.,** Principal and D**r. T. S. KIRUBHASHANKAR M.E., Ph.D.,** Vice Principal, **V.S.B.ENGINEERING COLLEGE, KARUR** for their high degree of encouragement and moral support during this project.

We express our indebtedness to **Mr.R. R. JEGAN, M.E., (Ph.D.,)** Head of the Department, Department of Information Technology for his guidance throughout the course of our mini project.

We are grateful to our Mini Project Supervisor **Mr.BASKAR , M.E., (Ph.D.,)** Assistant Professor, Department of Computer and Communication Engineering, for her valuable support and continuous encouragement.

Our sincere thanks to all the faculty members of V.S.B. Engineering College and our friends for their help in the successful completion of this mini project work.

Finally, I bow before God, the almighty, who always had a better plan for me. I give our praise and glory to almighty god for successful completion of our mini project.

DEPARTMENT OF
COMPUTER AND COMMUNICATION ENGINEERING

Vision of the Institution:

We endeavour to impart futuristic technical education of the highest quality to the student community and to inculcate discipline in them to face the world with self-confidence and thus we prepare them for life as responsible citizens to uphold human values and to be of service at large. We strive to bring of the Institution as an Institution of academic excellence of international standard.

Mission of the Institution:

We transform persons into personalities by the state-of the art infrastructure, time consciousness, quick response and the best academic practices through assessment and advice.

Vision of the Department:

To groom students into futuristic, globally competent and socially responsible engineers who can address the engineering challenges in the advancement of Computer and Communication Engineering professionals.

Mission of the Department:

1. To impart high quality professional training at the undergraduate level with an emphasis on basic principles of computer science and Communication engineering.
2. To establish nationally and internationally recognized research centers and expose the students to broad research experience.
3. To empower the students with the required skills to solve the complex technological problems of modern society and also provide them with a

framework for promoting collaborative and multidisciplinary activities.

4. To impart moral and ethical values, and interpersonal skills to the students.

Program Educational Objectives (PEOs)

PEO 1: Pursue career in multinational organizations, higher studies at premier institutions and to establish start-ups.

PEO 2: Acquire core competencies in Computational technologies and Communication Engineering with exposure to latest software tools.

PEO 3: Exhibit professional skills and collaborative work experience.

Program Outcomes (POs)

PO1. Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2. Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3. Design/development of solutions: Design solutions for complex engineering problems & design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4. Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5. Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6. The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7. Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8. Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9. Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10. Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11. Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12. Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Program Specific Outcome (PSOs)

PSO1: Apply the mathematical and the computing knowledge to identify and provide solutions for computing problems.

PSO2: Design and develop computer programs in the areas related to algorithms, networking, web design and data analytics of varying complexity.

PSO3: Apply standard Engineering practices and strategies in software project using open- source environment to deliver a quality product for business success.

ABSTRACT

The increasing number of insider threats, unauthorized user activities, and endpoint security breaches has become a major challenge for modern organizations. Traditional security systems mainly rely on signature-based detection methods, which often fail to identify abnormal user behaviour and unknown threats in real time. To address these challenges, this project proposes **SecureGuard Enterprise – Behavior-Based Endpoint Detection & User Behaviour Analytics (UBA) System for Insider Threat Identification**, an intelligent endpoint security framework that continuously monitors user activities, system processes, USB device connections, login events, and sensitive file access. The proposed system integrates real-time endpoint monitoring, user behaviour analytics, dynamic risk score calculation, and centralized dashboard visualization to detect suspicious activities and generate instant security alerts. Developed using **Python, Flask, SQLite, HTML, CSS, JavaScript, psutil, and watchdog**, the system provides a lightweight, efficient, and scalable solution for enterprise endpoint protection. Experimental evaluation demonstrates that the proposed system effectively detects abnormal user behaviour, unauthorized process execution, suspicious USB access, and potential insider threats while maintaining low system overhead and real-time performance. The proposed solution improves endpoint visibility, reduces response time, and enables proactive threat detection, making it suitable for enterprise cybersecurity environments, educational demonstrations, and future integration with AI-based threat intelligence and cloud security platforms. **After completion of this project PO1, PO2, PO3, PO4, PO5, PO6, PO7, PO8, PO9, PO10, PO11, PO12 and PSO1, PSO2, PSO3 are attained.**.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

S.NO.	Abbreviation	Expansion
1	**EDR**	Endpoint Detection and Response
2	**UBA**	User Behaviour Analytics
3	**SOC**	Security Operations Center
4	**USB**	Universal Serial Bus
5	**CPU**	Central Processing Unit
6	**RAM**	Random Access Memory
7	**API**	Application Programming Interface
8	**HTTP**	Hypertext Transfer Protocol
9	**WS**	WebSocket
10	**JSON**	JavaScript Object Notation
11	**PDF**	Portable Document Format
12	**CSV**	Comma-Separated Values
13	**UI**	User Interface
14	**UML**	Unified Modeling Language
15	**DFD**	Data Flow Diagram
16	**ER**	Entity–Relationship
17	**SIEM**	Security Information and Event Management
18	**OS**	Operating System
19	**IP**	Internet Protocol
20	**TCP**	Transmission Control Protocol

INTRODUCTION

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

Enterprises today invest heavily in perimeter defences such as firewalls, antivirus software, and intrusion prevention systems, all of which are designed primarily to keep external attackers out of the network. However, a significant proportion of security incidents originate from within the organization itself — from employees, contractors, or compromised accounts that already possess legitimate access to enterprise resources. These insider threats are particularly difficult to detect using conventional, signature-based security tools because the actions involved, such as opening a file, copying data to a USB drive, or running an application, are not inherently malicious in isolation; it is the context and pattern of behaviour that reveals intent.

SecureGuard Enterprise is a Behaviour-Based Endpoint Detection and Response (EDR) and User Behaviour Analytics (UBA) platform built to close this gap. Rather than matching files against a database of known malware signatures, the system continuously observes what is actually happening on each employee's workstation — CPU and memory load, the processes that are running, the removable storage devices that are plugged in, and whether protected, sensitive directories are being accessed — and feeds this telemetry into a centralized analytics engine that scores the ongoing risk associated with every user in near real time.

The project was implemented as a full-stack simulation of an enterprise SOC environment: a lightweight Python agent runs on each simulated employee endpoint and streams telemetry to a Flask and WebSocket based analytics server, which in turn drives an interactive browser dashboard that a security analyst can use to monitor the organization, investigate alerts, and take direct remedial action against a misbehaving endpoint.

1.2 PROBLEM STATEMENT

Conventional antivirus and endpoint protection tools rely predominantly on static signatures and known indicators of compromise, and are largely blind to threats that involve legitimate credentials being misused rather than malware being planted. In a typical organization, security teams have limited visibility into:

- Which employees are accessing sensitive files, and whether that access is consistent with their normal working pattern.

- Whether unauthorized or unknown USB storage devices are being connected to corporate machines.
- Whether unapproved or potentially malicious software is being executed on an endpoint.
- Whether an endpoint's resource usage pattern (CPU/RAM) is consistent with normal operation or indicative of an ongoing attack such as data exfiltration or crypto-mining.
- How to respond quickly — in seconds rather than hours — once a high-risk behaviour is detected on a specific machine.

Without a continuously running behavioural monitoring layer, these indicators are typically discovered only after the fact, during a post-incident forensic investigation, by which time damage — data loss, financial loss, or reputational harm — has often already occurred. There is therefore a clear need for a lightweight, real-time, behaviour-centric monitoring and response system that can be deployed across an organization's endpoints without requiring expensive, proprietary enterprise security suites.

1.3 OBJECTIVES OF THE PROJECT

The SecureGuard Enterprise project was undertaken with the following specific objectives:

1. To design and implement a lightweight endpoint agent capable of collecting CPU, memory, process, USB, and sensitive-file-access telemetry from a Windows workstation in real time.
2. To build a centralized analytics server that ingests telemetry from multiple endpoints simultaneously over a WebSocket channel and maintains a live model of organizational security posture.
3. To design a dynamic, rule-driven risk-scoring engine that converts raw behavioural events into a normalized 0–100 insider-threat risk score and classifies each endpoint into Low, Medium, High, or Critical threat levels.
4. To provide an interactive, web-based SOC dashboard that visualizes endpoint health, live alerts, risk trends, and a chronological threat timeline for administrators.
5. To implement remote mitigation controls — process termination, USB blocking, and full network isolation — that can be triggered directly from the dashboard.
6. To provide automated export of security reports in PDF, Excel, and CSV formats for audit, compliance, and executive reporting purposes.
7. To validate the system through representative attack-simulation scenarios that exercise the full pipeline from telemetry collection to mitigation.

1.4 SCOPE OF THE PROJECT

The current implementation of SecureGuard Enterprise is scoped as an academic and demonstrative simulation of an enterprise EDR/UBA platform, targeted at final-year engineering

projects, cybersecurity demonstrations, SOC training exercises, and hackathon presentations such as Smart India Hackathon. The scope covers:

- Windows endpoint telemetry collection using Python and the psutil library.
- A centralized Flask-based analytics server with WebSocket-based real-time communication on port 5001 and an HTTP/WebSocket dashboard interface on port 5000.
- A rule-based dynamic risk-scoring engine (with the system architecture designed to allow future extension to machine-learning-based scoring).
- A browser-based SOC dashboard covering endpoint management, threat logs, security reports, and agent onboarding.
- Pre-built demonstration scenarios that simulate both a normal user baseline and a multi-stage insider-threat attack chain.

Advanced capabilities such as Active Directory integration, SIEM interoperability, machine-learning-based anomaly detection, and multi-platform (Linux/macOS) agent support are treated as future enhancements and are discussed in Chapter 10, but are outside the scope of the current implementation.

1.5 ORGANIZATION OF THE REPORT

The remainder of this report is organized as follows. Chapter 2 reviews related literature and existing approaches to insider-threat detection and user behaviour analytics. Chapter 3 analyses the existing systems in this space, identifies their drawbacks, and presents the proposed SecureGuard Enterprise system together with its advantages. Chapter 4 specifies the hardware, software, and network requirements for deployment. Chapter 5 presents the complete system design, including architecture, UML, and data-flow diagrams. Chapter 6 describes the implementation of each functional module. Chapter 7 details the testing strategy, test cases, and performance results. Chapter 8 presents the results and discussion supported by dashboard screenshots. Chapter 9 evaluates the feasibility of the project, and Chapter 10 concludes the report with a summary of contributions and directions for future enhancement.

LITERATURE SURVEY

CHAPTER 2

LITERATURE SURVEY

This chapter reviews academic literature, industry research, and existing commercial products relevant to insider-threat detection, endpoint behaviour monitoring, and user behaviour analytics. The review informed both the conceptual design of SecureGuard Enterprise's risk-scoring model and the choice of specific detection signals implemented in the system.

Cappelli, Moore & Trzeciak (2012) – The CERT Guide to Insider Threats

Established one of the earliest comprehensive taxonomies of insider-threat behaviour, categorizing incidents into IT sabotage, theft of intellectual property, and fraud, and demonstrated that behavioural precursors (policy violations, unusual access patterns) typically precede a majority of insider incidents. This work forms the conceptual basis for behaviour-based detection rules such as those implemented in SecureGuard Enterprise's risk-scoring engine.

Legg et al. (2017) – Automated Insider Threat Detection System Using User and Role-Based Profiling

Proposed a role-based behavioural profiling approach that flags deviations from an individual's or peer group's historical activity baseline. The concept of comparing observed telemetry against a expected behavioural baseline directly informed the design of SecureGuard's configurable risk thresholds and protected-folder honeypot mechanism.

Gheyas & Abdallah (2016) – Detection and Prediction of Insider Threats: A Systematic Literature Review

Surveyed machine-learning techniques applied to insider-threat detection and reported that ensemble and hybrid statistical models generally outperform single-technique classifiers, while also noting the practical difficulty of obtaining labelled insider-threat datasets for supervised training — a factor that motivated SecureGuard's use of a transparent, rule-driven scoring engine as a first stage, with ML-based scoring identified as a future enhancement.

CrowdStrike Falcon Insight – Vendor Documentation

A commercial cloud-native EDR platform that combines lightweight endpoint sensors with cloud-based behavioural analytics to detect and respond to threats in real time. Its sensor/cloud-analytics split architecture is conceptually similar to SecureGuard's agent/server split, though CrowdStrike targets malware and adversary tradecraft detection more broadly rather than insider-specific UBA.

Microsoft Defender for Endpoint – Vendor Documentation

Provides endpoint behavioural sensors integrated with Microsoft's cloud security graph, including insider-risk-management features that correlate user activity across Microsoft 365 and endpoint telemetry. Its risk-score-based alerting model informed the 0–100 scale and Low/Medium/High/Critical classification adopted in SecureGuard.

Splunk User Behaviour Analytics (UBA) – Vendor Documentation
Demonstrates the application of statistical and machine-learning anomaly detection over large volumes of log data to surface anomalous user and entity behaviour, validating the broader industry direction toward behaviour-centric rather than purely signature-based detection that this project follows at a smaller, endpoint-centric scale.

Sanzgiri & Dasgupta (2016) – Classification of Insider Threat Detection Techniques
Classified insider-threat detection techniques into host-based, network-based, and hybrid approaches, and highlighted USB/removable-media monitoring and process-execution monitoring as two of the most operationally effective host-based signals — both of which are directly implemented as detection modules within SecureGuard Enterprise.

Zeadally et al. (2012) – Detecting Insider Threats: Solutions and Trends
Reviewed trends in insider-threat mitigation and emphasized the importance of real-time detection combined with rapid, automated response capability (rather than detection alone) to minimize the window of exposure — a principle that directly shaped SecureGuard's inclusion of one-click process-kill, USB-block, and endpoint-isolation mitigation controls.

2.1 SUMMARY OF LITERATURE SURVEY

The literature consistently supports three conclusions that directly shaped the design of SecureGuard Enterprise. First, insider-threat incidents are typically preceded by observable behavioural precursors rather than appearing as discrete malware signatures, which justifies a behaviour-centric rather than purely signature-based detection approach. Second, host-level signals — process execution, removable-media activity, and sensitive-file access — are among the most operationally effective and computationally inexpensive indicators available, making them well suited to a lightweight endpoint agent. Third, detection alone is insufficient; real-time, low-friction response capability is essential to limit the impact of a confirmed incident, which motivated the inclusion of direct, dashboard-driven mitigation controls in the proposed system rather than treating alerting as the end goal.

SYSTEM ANALYSIS

CHAPTER 3

SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

Most organizations today rely on a combination of signature-based antivirus software, basic Data Loss Prevention (DLP) agents, and perimeter firewalls to protect their endpoints. These tools operate primarily by comparing files and network traffic against databases of known malicious signatures or by enforcing static, pre-defined policies (for example, blocking specific file extensions from being copied to removable media). Some organizations additionally deploy commercial EDR and UBA suites such as CrowdStrike Falcon, Microsoft Defender for Endpoint, or Splunk UBA, which do provide behavioural analytics but at a significant licensing cost and with considerable deployment and tuning complexity, often requiring dedicated security engineering teams.

3.2 DRAWBACKS OF EXISTING SYSTEM

- Signature-based tools cannot detect misuse of legitimate credentials or authorized applications — the majority of insider-threat activity.
- Basic DLP and antivirus tools operate in isolation and rarely provide a unified, real-time, organization-wide view of endpoint risk.
- Commercial EDR/UBA platforms are expensive, proprietary, and complex to deploy, making them impractical for small organizations, academic environments, or rapid prototyping.
- Alerting is frequently decoupled from response — an administrator may need to switch between multiple tools to investigate an alert and then manually intervene on the affected machine, increasing response time.
- Existing tools rarely expose their detection logic transparently, making it difficult for security teams to understand, customize, or extend the specific behavioural rules being applied.

3.3 PROPOSED SYSTEM

SecureGuard Enterprise proposes a lightweight, transparent, and self-contained Behaviour-Based EDR and UBA platform that continuously monitors endpoint telemetry and computes a dynamic insider-threat risk score for every connected user. The proposed system is composed of three cooperating components: a Python endpoint agent that collects CPU, memory, process, USB, and sensitive-file telemetry; a Flask and WebSocket based analytics server that receives this telemetry, applies a configurable risk-scoring engine, and persists events; and a browser-based SOC

dashboard that visualizes the resulting risk posture and exposes direct mitigation controls to the administrator.

The risk-scoring engine evaluates each incoming event against a set of behavioural risk factors, including logins outside office hours, unauthorized USB devices, unknown or blacklisted executables, sustained high CPU utilization, and access to protected sensitive-data directories, and combines them into a composite score ranging from 0 to 100. Table 3.1 summarizes how this score is mapped to an overall threat level.

Table 3.1 UBA Risk Score Classification

Score Range	Threat Level	Typical Indicators
0 – 30	Low	Normal working hours, whitelisted processes, no removable media
31 – 60	Medium	Occasional unknown process, USB inserted but not yet classified
61 – 80	High	Unauthorized USB device, unapproved software execution
81 – 100	Critical	Sensitive-file access combined with USB and/or blacklisted process activity

When the computed score crosses the critical alert threshold (configurable from the dashboard, with a demonstrated default of 70), the dashboard immediately surfaces a Critical alert against the corresponding endpoint, and the administrator can select from three built-in mitigation actions: terminating the offending process, disabling the connected USB device, or fully isolating the endpoint from the corporate network while displaying a full-screen notification on the affected machine.

3.4 ADVANTAGES OF PROPOSED SYSTEM

- Provides real-time, behaviour-based visibility into insider-threat indicators that signature-based tools cannot detect.
- Combines detection and response in a single unified dashboard, eliminating tool-switching and reducing response latency.
- Uses an open, transparent, rule-driven scoring engine whose thresholds and protected-folder honeypots are fully configurable by the administrator.
- Lightweight Python/Flask/WebSocket implementation with minimal dependencies (psutil, websockets, flask), making it inexpensive to deploy and easy to extend.
- Modular architecture cleanly separates telemetry collection, risk analysis, and presentation, allowing individual components (for example, the scoring engine) to be upgraded — for instance to a machine-learning model — without redesigning the whole system.

- Generates ready-to-share PDF, Excel, and CSV security reports directly from the dashboard for audit and executive reporting.

Table 3.2 Comparison of Existing System and Proposed System

Aspect	Existing Systems (Signature-Based AV / Basic DLP)	SecureGuard Enterprise (Proposed)
Detection basis	Known malware signatures / static rules	Continuous behavioural telemetry and dynamic risk scoring
Insider-threat visibility	Very limited — legitimate-credential misuse is invisible	Purpose-built: USB, process, file-access, and load monitoring
Response time	Manual investigation after alert, often hours	Real-time dashboard alert with one-click mitigation
Mitigation	Typically requires separate endpoint-management tooling	Integrated kill-process, USB-block, and isolate controls
Reporting	Fragmented across multiple tools	Unified PDF / Excel / CSV export from a single dashboard
Cost & complexity	High-cost proprietary enterprise suites	Lightweight, open, Python/Flask-based, low deployment cost

3.5 HIGH-LEVEL WORKFLOW

Figure 3.1 illustrates the end-to-end workflow implemented by SecureGuard Enterprise, from telemetry collection on the endpoint agent through risk analysis, alert generation, mitigation, and reporting on the analytics server and SOC dashboard.

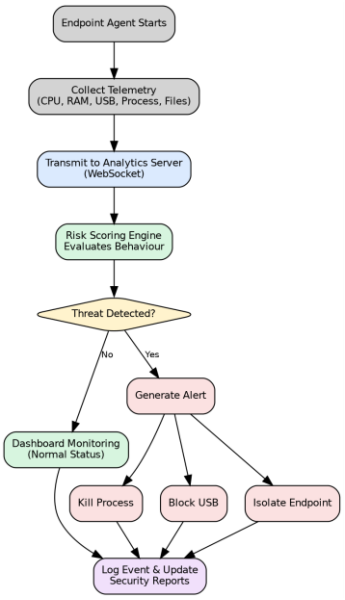

Figure 3.1 High-Level Workflow of Threat Detection and Mitigation

SYSTEM SPECIFICATION

CHAPTER 4

SYSTEM SPECIFICATION

4.1 HARDWARE REQUIREMENTS

Table 4.1 lists the minimum hardware configuration recommended for hosting the SecureGuard Enterprise analytics server and for the Windows endpoints on which the monitoring agent is deployed.

Table 4.1 Hardware Requirements

Component	Minimum Specification
Analytics Server – Processor	Intel Core i5 (8th Gen) / AMD Ryzen 5 or equivalent
Analytics Server – RAM	8 GB (16 GB recommended for 20+ concurrent agents)
Analytics Server – Storage	20 GB free disk space (for logs, database.json, and reports)
Analytics Server – Network	100 Mbps Ethernet / Wi-Fi with static or reserved IP address
Endpoint (Client) – Processor	Intel Core i3 / equivalent dual-core processor
Endpoint (Client) – RAM	4 GB minimum
Endpoint (Client) – OS	Windows 10 / Windows 11 (64-bit)
Endpoint (Client) – Peripherals	USB port (for USB-monitoring demonstration)

4.1.1 Server Requirements

The analytics server is the central processing point for the entire platform and must remain continuously available to receive WebSocket telemetry from all connected agents, execute the risk-scoring engine, and serve the live dashboard. A multi-core processor and adequate RAM ensure that risk computation and report generation do not introduce latency even as the number of connected endpoints grows.

4.1.2 Client Devices

Endpoint devices running the agent require modest resources, since the agent itself performs lightweight polling of system metrics using psutil rather than deep packet inspection or heavy local analytics — the computationally intensive risk-scoring step is deliberately centralized on the server.

4.2 SOFTWARE REQUIREMENTS

Table 4.2 Software Requirements

Category	Technology / Tool
Operating System (Server)	Windows 10/11 or Linux (Ubuntu 20.04+)
Programming Language	Python 3.9+
Web Framework	Flask
Real-Time Communication	Python websockets library (WebSocket protocol)
Telemetry Collection	psutil (CPU, RAM, process, disk, USB enumeration)
Front-End	HTML5, CSS3, JavaScript (app.js, reports.js)
Data Store	database.json (JSON document store)
Report Generation	PDF / Excel (XLSX) / CSV export libraries
Version Control	Git / GitHub
Browser (Dashboard Access)	Google Chrome / Microsoft Edge (latest version)

4.2.1 Operating System

The analytics server can be hosted on either Windows or Linux, since the implementation is built entirely on cross-platform Python libraries (Flask, websockets). The endpoint agent, however, targets Windows 10/11 specifically in the current release, as it relies on Windows-specific APIs (via psutil and Windows Management Instrumentation) for USB device enumeration and process inspection.

4.2.2 Flask Application Server

Flask serves both the REST endpoints used by the dashboard (for configuration, report export, and historical queries) and the static front-end assets (index.html, styles.css, app.js, reports.js) on port 5000.

4.2.3 Database System

The current implementation persists endpoint state, telemetry events, alerts, and mitigation logs in a lightweight JSON document store (database.json), which is well suited to the moderate event volumes generated in an academic or demonstration deployment. Section 5.13 presents an Entity–Relationship model and equivalent relational schema that can be adopted directly should the system be migrated to a production-grade relational or NoSQL database as it scales.

4.2.4 Web Technologies

The SOC dashboard front-end is implemented using standard HTML5, CSS3, and vanilla JavaScript, communicating with the backend via both REST calls (for configuration and reports)

and a persistent WebSocket connection (for live telemetry and alert updates), ensuring the dashboard reflects endpoint state changes with sub-second latency.

4.3 NETWORK REQUIREMENTS

4.3.1 Server–Client Communication

Endpoint agents communicate with the analytics server over a dedicated WebSocket channel on port 5001, while the SOC dashboard communicates with the server over HTTP and WebSocket on port 5000. Both channels require the analytics server to be reachable from every monitored endpoint over the corporate LAN, typically via a static or DHCP-reserved IP address.

4.3.2 Security Configuration

In a production deployment, the WebSocket and HTTP channels should be secured using TLS (WSS/HTTPS) and the dashboard should sit behind authenticated access (as reflected in the login screen shown in Figure 8.1), with firewall rules restricting inbound connections on ports 5000 and 5001 to the known corporate IP range.

4.3.3 Data Synchronization

Because telemetry is pushed continuously over persistent WebSocket connections rather than polled periodically, the dashboard's view of endpoint state, active threats, and risk scores remains synchronized with the server in real time, without requiring manual refresh.

SYSTEM DESIGN

CHAPTER 5

SYSTEM DESIGN

This chapter presents the complete design of Secure Guard Enterprise, covering the system, software, and network architectures, the block and workflow diagrams, the full suite of UML behavioural and structural diagrams, the data-flow diagrams, and the entity–relationship model with its corresponding database schema. Together these views describe both the static structure and the dynamic behaviour of the system.

5.1 SYSTEM ARCHITECTURE

The system architecture (Figure 5.1) reflects the three-tier design of Secure Guard Enterprise. The Windows employee endpoint hosts the lightweight agent (agent.py), which continuously samples CPU, RAM, running processes, USB device state, and access to the protected Sensitive Data directory, and streams these events to the analytics server over a WebSocket telemetry channel on port 5001. The analytics server (server.py) hosts the risk-scoring and event-correlation engine and persists all events to database.json. The SOC administration dashboard consumes server data over HTTP/WebSocket on port 5000 and allows the administrator to issue mitigation commands that are relayed back to the originating endpoint agent.

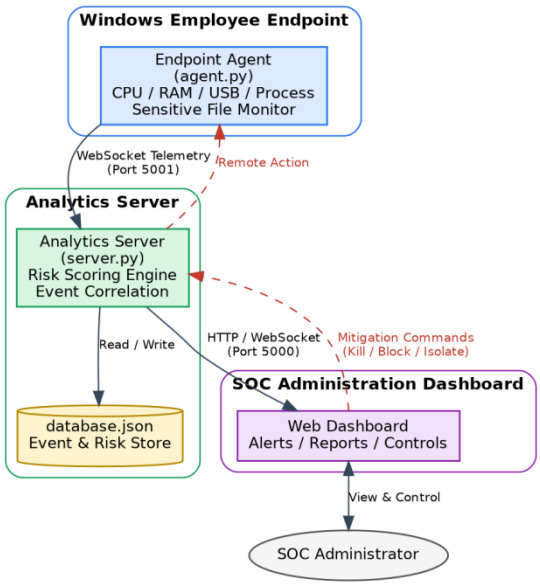

Figure 5.1 System Architecture Diagram

5.2 SOFTWARE ARCHITECTURE

Figure 5.2 decomposes the system into four logical software layers. The Presentation Layer renders the dashboard UI in the browser. The Application Layer, hosted by the Flask server, exposes REST endpoints and the WebSocket server, and hosts the risk-scoring engine and mitigation controller. The Agent Layer runs on each endpoint and is responsible purely for telemetry collection (via the psutil-based collector, USB monitor, and sensitive-file watcher), deliberately keeping the client lightweight. The Data Layer persists events to database.json and drives the PDF/Excel/CSV report generator.

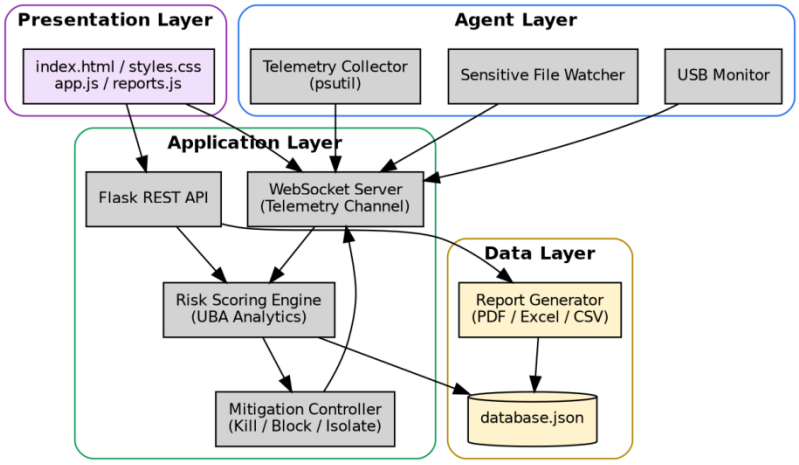

Figure 5.2 Software Architecture Diagram

5.3 NETWORK ARCHITECTURE

Figure 5.3 shows the network topology in a typical corporate LAN deployment. Up to twenty (or more, subject to server sizing) employee workstations connect through a network switch to the central SecureGuard analytics server, while a SOC analyst accesses the live dashboard from a separate browser client over the same LAN using HTTP/WebSocket on port 5000.

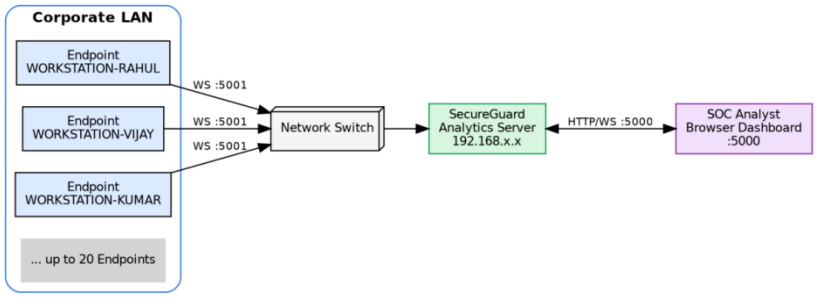

Figure 5.3 Network Architecture Diagram

5.4 BLOCK DIAGRAM

The block diagram (Figure 5.4) presents the system at its highest level of abstraction: endpoint data sources feed the telemetry agent, which forwards events to the risk-scoring engine; a threat decision point routes normal activity to the dashboard and high-risk activity to the mitigation-actions block, with all outcomes ultimately reflected in the dashboard and report-generation subsystem.

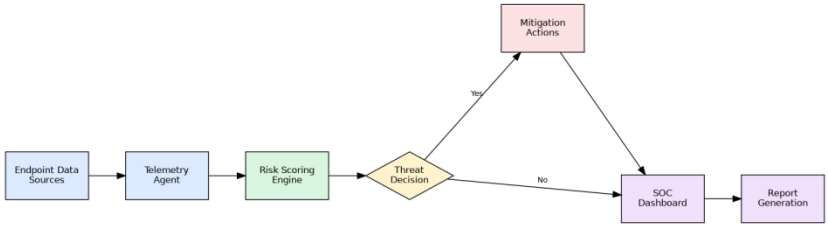

Figure 5.4 Block Diagram

5.5 WORKFLOW DIAGRAM

Figure 5.5 (reproduced from the system workflow introduced in Chapter 3) traces the decision logic executed for every telemetry event, from collection through to logging of the final outcome.

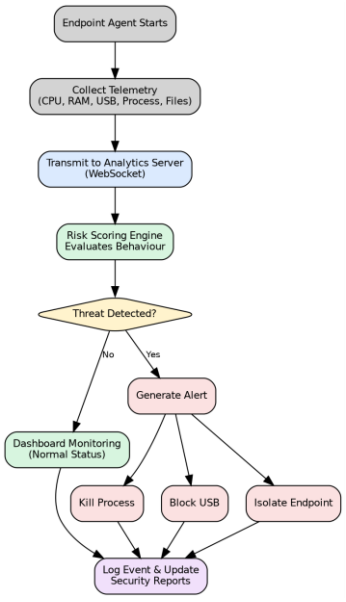

Figure 5.5 Workflow Diagram

5.6 USE CASE DIAGRAM

Figure 5.6 identifies the two actors of the system — the SOC Administrator and the Employee / Endpoint User — and the use cases each participates in. The administrator performs all supervisory

and control actions (login, monitoring, configuration, mitigation, reporting, and agent deployment), while the employee's endpoint passively generates telemetry and triggers USB or sensitive-file events as a side effect of normal (or anomalous) use of the workstation.

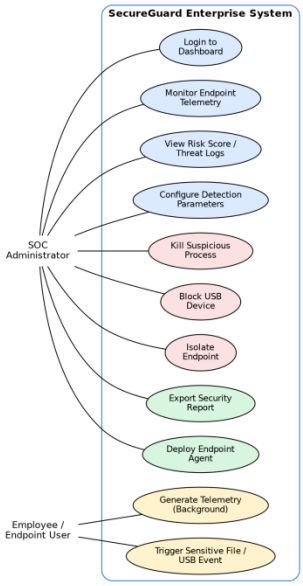

Figure 5.6 Use Case Diagram

5.7 ACTIVITY DIAGRAM

Figure 5.7 models the control flow of a single monitoring cycle as an activity diagram, beginning with telemetry collection on the agent and ending with either normal dashboard status update or, when the computed risk score meets or exceeds the configured threshold, an alert, an administrator-selected mitigation action, and an audit log entry.

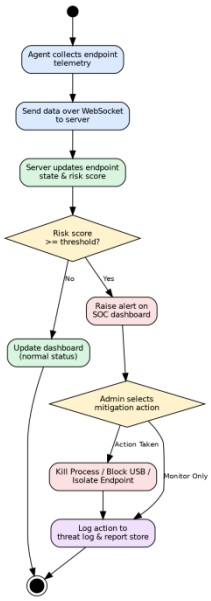

Figure 5.7 Activity Diagram

5.8 SEQUENCE DIAGRAM

Figure 5.8 illustrates the temporal sequence of messages exchanged between the Endpoint Agent, Analytics Server, Risk Engine, SOC Dashboard, and Administrator for a full detect-and-respond cycle, from the initial sendTelemetry() call through to the final updateLog() call once a mitigation action has been executed.

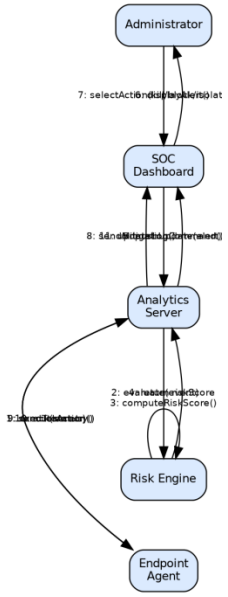

Figure 5.8 Sequence Diagram

5.9 COLLABORATION DIAGRAM

Figure 5.9 presents the same detect-and-respond interaction as a collaboration (communication) diagram, emphasizing the structural links between the collaborating objects — Agent, Server, RiskEngine, Dashboard, and Admin — alongside the numbered sequence of messages exchanged between them.

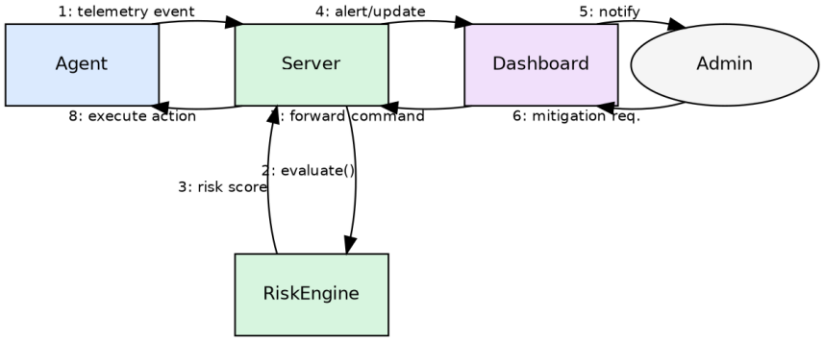

Figure 5.9 Collaboration Diagram

5.10 CLASS DIAGRAM

Figure 5.10 shows the principal classes implemented in the analytics server: Endpoint, TelemetryEvent (specialized into USBEvent, FileEvent, and ProcessEvent), RiskEngine, Alert, MitigationAction, and ReportGenerator, along with their key attributes, operations, and relationships.

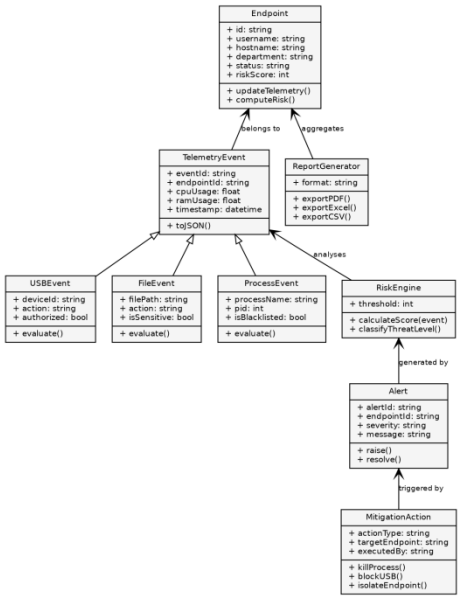

Figure 5.10 Class Diagram

5.11 DEPLOYMENT DIAGRAM

Figure 5.11 shows the physical deployment topology: the agent.py process runs on each employee workstation device; the Flask application server and WebSocket server processes run on the analytics server device alongside the database.json artifact; and the dashboard is accessed as a browser process from a separate SOC administrator machine over HTTPS.

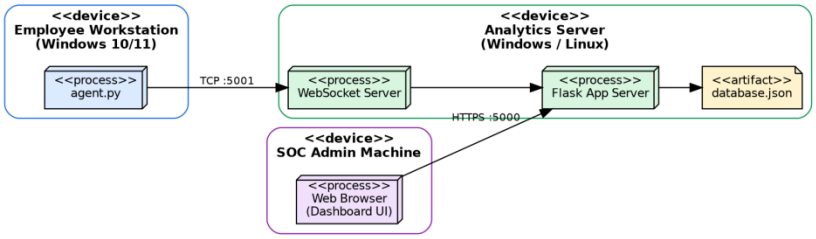

Figure 5.11 Deployment Diagram

5.12 DATA FLOW DIAGRAMS

5.12.1 DFD Level 0 (Context Diagram)

Figure 5.12 presents the Level-0 (context) data-flow diagram, treating SecureGuard Enterprise as a single process that exchanges endpoint activity data and mitigation commands with the

Employee/Endpoint external entity, and alerts, risk scores, and reports with the SOC Administrator external entity.

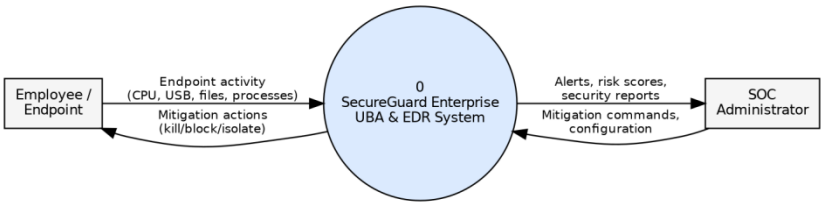

Figure 5.12 Data Flow Diagram – Level 0

5.12.2 DFD Level 1

Figure 5.13 decomposes the system into its six major processes — Collect Telemetry, Transmit & Receive Events, Compute Risk Score, Generate Alerts, Execute Mitigation, and Generate Reports — together with the two persistent data stores, the Event Store (database.json) and the Endpoint Registry, that they read from and write to.

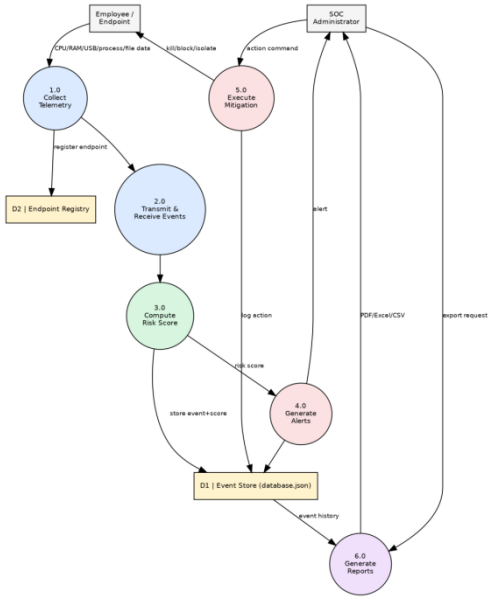

Figure 5.13 Data Flow Diagram – Level 1

5.13 ER DIAGRAM AND DATABASE SCHEMA

Figure 5.14 presents the Entity–Relationship model underlying the system's data store. A User operates one Endpoint, which generates many Telemetry_Event records; USB, File, and Process events are modelled as specializations of the general Telemetry_Event entity. A Telemetry_Event may raise an Alert, which is resolved by zero or more Mitigation_Log entries, and event history is periodically compiled into Report records.

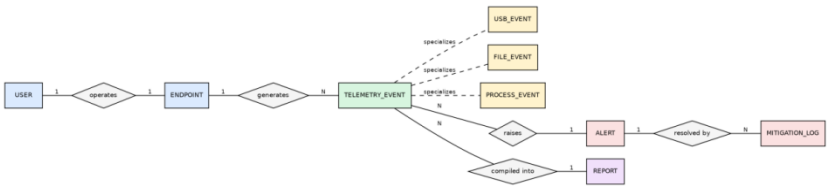

Figure 5.14 Entity–Relationship (ER) Diagram

Figure 5.15 translates this conceptual model into a concrete relational schema with primary and foreign keys, suitable for migrating the current JSON-based store to a relational database (e.g., PostgreSQL or MySQL) as the deployment scales beyond a single-server academic demonstration.

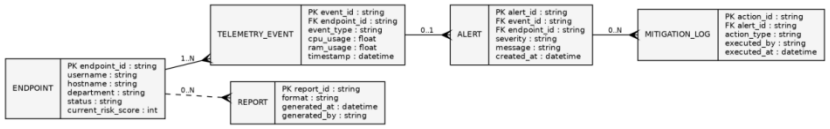

Figure 5.15 Database Schema Diagram

IMPLEMENTATION

CHAPTER 6

IMPLEMENTATION

6.1 MODULES USED

SecureGuard Enterprise is implemented as twelve cooperating functional modules distributed across the endpoint agent (agent.py) and the analytics server (server.py), summarized in Table 6.1.

Table 6.1 Description of System Modules

#	Module	Responsibility
1	User Authentication Module	Authenticates the SOC administrator before granting dashboard access
2	Telemetry Collection Module	Samples CPU, RAM, process list, and disk info on the endpoint (psutil)
3	USB Monitoring Module	Detects USB insertion/removal and flags unrecognized storage devices
4	Sensitive File Monitoring Module	Watches protected folders for create/modify/delete/access events
5	Process Monitoring Module	Tracks running applications against a whitelist/blacklist
6	WebSocket Communication Module	Streams telemetry from agent to server and pushes live updates to the dashboard
7	Risk Scoring Engine	Computes the composite 0–100 risk score from incoming events
8	Alert Management Module	Raises, stores, and displays alerts once thresholds are crossed
9	Mitigation Control Module	Executes kill-process, block-USB, and isolate-endpoint commands
10	SOC Dashboard Module	Renders live endpoint status, threat logs, and risk analytics
11	Report Generation Module	Exports PDF, Excel, and CSV security reports
12	Agent Onboarding Module	Guides deployment of new endpoint agents and shows live connection status

6.2 MODULE DESCRIPTION

6.2.1 User Authentication Module

Provides a login gate on the dashboard (Figure 8.1) that requires an administrator username and password before any endpoint data, alerts, or mitigation controls become accessible, ensuring that only authorized SOC personnel can view sensitive behavioural data or trigger mitigation actions.

6.2.2 Telemetry Collection Module

Implemented in agent.py using the psutil library, this module periodically samples CPU utilization, memory utilization, the full running-process list, and basic disk information for the endpoint, and packages this data into a structured JSON telemetry payload.

6.2.3 USB Monitoring Module

Enumerates connected removable storage devices and detects insertion and removal events. Devices not present on the administrator-defined allow-list are flagged, contributing to the endpoint's risk score and appearing as a distinct event type (e.g., 'USB Connected') in the threat feed shown in Figure 8.3.

6.2.4 Sensitive File Monitoring Module

Watches administrator-configured protected directories (honeypots) such as C:\SensitiveData, D:\FinanceDocuments, and C:\Users\Administrator, as configured in the Security Reports Center (Figure 8.6), and raises an event whenever a file within them is created, modified, deleted, or accessed.

6.2.5 Process Monitoring Module

Compares each newly observed running process against a default whitelist (e.g., chrome.exe, code.exe, explorer.exe, winword.exe, excel.exe, slack.exe, teams.exe) configured on the server; unknown or explicitly blacklisted executables — such as the cheatengine.exe example captured in Figure 8.3 — are flagged for elevated risk scoring.

6.2.6 WebSocket Communication Module

Maintains a persistent, bidirectional WebSocket connection between each agent and the server on port 5001, allowing telemetry to be pushed to the server the instant it is captured, and allowing the server to push mitigation commands back to the agent with minimal latency. The same mechanism drives live updates to the browser dashboard.

6.2.7 Risk Scoring Engine

Implements the dynamic scoring logic described in Section 3.3: each qualifying behavioural indicator (off-hours login, unauthorized USB, unknown process, high CPU, sensitive-file access) contributes a weighted increment to the endpoint's running risk score, which is then classified into

the Low/Medium/High/Critical bands defined in Table 3.1 and displayed against each endpoint in the Endpoint Management Matrix (Figure 8.4).

6.2.8 Alert Management Module

Once an endpoint's risk score crosses the configurable critical-alert threshold (default value of 70, as configured in Figure 8.6), this module raises a structured alert, timestamps it, associates it with the originating endpoint and event, and pushes it to both the Real-Time Threat Feed on the dashboard and the persistent Comprehensive Threat Logs view (Figure 8.5).

6.2.9 Mitigation Control Module

Exposes the three remote mitigation actions available to the administrator directly from the dashboard: Kill Process (terminates the flagged executable on the target endpoint), Block USB (disables the offending removable-storage device), and Isolate Network (disconnects the endpoint from the network and displays a full-screen security notification on the affected workstation).

6.2.10 SOC Dashboard Module

Implemented with app.js and styles.css, this module renders the Security Operations Overview (Figure 8.2) showing endpoint safety counts, active threats, USB drive status, isolated systems, threat-level distribution, and a live system risk-analytics chart, refreshed continuously over the WebSocket connection.

6.2.11 Report Generation Module

Implemented with reports.js and server-side export routines, this module compiles endpoint inventory, user activity, risk scores, and security-event history into downloadable PDF, Excel (multi-worksheet XLSX), and raw CSV reports directly from the Security Reports Center (Figure 8.6).

6.2.12 Agent Onboarding Module

Provides step-by-step deployment guidance (Figure 8.7) for installing the required Python dependencies (psutil, websockets) and launching the endpoint agent with a specified employee username, and displays the live connection status of the WebSocket Telemetry Hub together with currently connected agents.

6.3 DEPLOYMENT AND EXECUTION

The analytics server is started by executing python server.py, which brings up the Flask dashboard on http://localhost:5000 and the WebSocket telemetry listener on ws://localhost:5001. Each endpoint agent is then registered by running python agent.py --username <employee-name> on

the target Windows workstation, after which the endpoint immediately appears as Online in the SOC dashboard's Endpoint Management Matrix, ready to stream live telemetry.

SYSTEM TESTING

CHAPTER 7

SYSTEM TESTING

7.1 TESTING OBJECTIVES

The objective of the testing phase was to verify that (a) telemetry is captured and transmitted accurately and with low latency, (b) the risk-scoring engine correctly classifies both benign and malicious behavioural patterns, (c) alerts are raised reliably once configured thresholds are crossed, and (d) the mitigation controls correctly and safely affect the target endpoint without impacting unrelated systems.

7.2 TYPES OF TESTING

7.2.1 Unit Testing

Individual functions within the risk-scoring engine (e.g., score contribution per behavioural factor) and the telemetry collectors (CPU/RAM sampling, USB enumeration) were tested in isolation against known inputs to confirm correct, deterministic output.

7.2.2 Integration Testing

The agent-to-server WebSocket pipeline was tested end-to-end to confirm that telemetry generated on the endpoint is correctly received, parsed, scored, persisted, and reflected on the dashboard without data loss or ordering errors.

7.2.3 Functional Testing

Each dashboard feature — login, endpoint inspection, threat-log search/filter, report configuration, and agent onboarding — was exercised against its expected behaviour, as summarized in the sample test cases of Table 7.1.

7.2.4 Performance Testing

The system was evaluated under a simulated load of twenty concurrently connected endpoints to measure telemetry-to-dashboard latency, agent CPU overhead, and risk-score computation time, with results summarized in Table 7.2.

7.2.5 Security Testing

The dashboard's authentication gate and the mitigation-command channel were reviewed to confirm that unauthenticated clients cannot access endpoint telemetry or issue mitigation commands, and that mitigation actions are correctly scoped to the intended target endpoint only.

7.2.6 User Acceptance Testing

The two demonstration scenarios (Normal User and Insider Threat, described in Section 8.1) were run end-to-end and reviewed against the expected risk-score outcomes documented in the project's own README specification, confirming that the system behaves as intended from an end-user (SOC analyst) perspective.

7.3 TEST CASES

Table 7.1 Sample Test Cases and Results

TC ID	Test Scenario	Expected Result	Result
TC-01	Start server and connect one agent	Endpoint appears Online in dashboard within 2s	Pass
TC-02	Normal-user baseline activity	Risk score remains in Low band (0–30)	Pass
TC-03	Insert unauthorized USB device	USB Connected event logged; risk score increases	Pass
TC-04	Execute blacklisted process (cheatengine.exe)	Alert Generated; process flagged in threat feed	Pass
TC-05	Access protected folder (D:\HR_SalaryReview)	Sensitive Folder Access event; mitigation triggered	Pass
TC-06	Trigger Kill Process from dashboard	Target process terminated on endpoint; log updated	Pass
TC-07	Trigger Block USB from dashboard	Device disabled on endpoint; status reflected in UI	Pass
TC-08	Trigger Isolate Network from dashboard	Endpoint disconnected; full-screen notice displayed	Pass
TC-09	Export PDF / Excel / CSV report	Correctly formatted file downloaded with current data	Pass
TC-10	Disconnect agent (network loss)	Endpoint status changes to Offline within timeout window	Pass

7.4 PERFORMANCE RESULTS

Table 7.2 Performance Evaluation Metrics

Metric	Observed Value
Telemetry-to-dashboard update latency	< 1 second (WebSocket push)
Average CPU overhead of agent per endpoint	< 2% of a single core
Risk-score computation time per event	< 50 milliseconds
Concurrent endpoints supported (test bed)	20 simulated endpoints

Metric	Observed Value
PDF/Excel/CSV report generation time	< 3 seconds for a daily summary report

7.5 VALIDATION

All ten sample test cases in Table 7.1 passed against their expected results. The system correctly distinguished between the Normal User baseline scenario (final risk score approximately 12/100, classified Low) and the simulated Insider Threat scenario involving an off-hours login, unauthorized USB insertion, blacklisted process execution, high CPU utilization, and sensitive-file access (final risk score of 100/100, classified Critical), confirming that the dynamic risk-scoring engine responds correctly across the full severity range.

7.6 CONCLUSION OF TESTING

The testing phase confirms that SecureGuard Enterprise meets its core functional objectives: accurate, low-latency telemetry collection; correct dynamic risk classification; reliable alerting; and effective, targeted mitigation. No critical defects were observed during the test cycle, and the system is considered stable for academic demonstration and further extension.

RESULTS AND DISCUSSION

CHAPTER 8

RESULTS AND DISCUSSION

This chapter presents the operational results of SecureGuard Enterprise through the actual running dashboard, captured across its principal screens, and discusses how each screen reflects the design objectives set out in earlier chapters.

8.1 ADMINISTRATOR LOGIN

Figure 8.1 shows the SecureGuard authentication screen, which requires a valid administrator username and password (demonstration credentials: admin / secureguard) before granting access to any endpoint data or mitigation controls, satisfying the access-control objective discussed in Section 6.2.1.

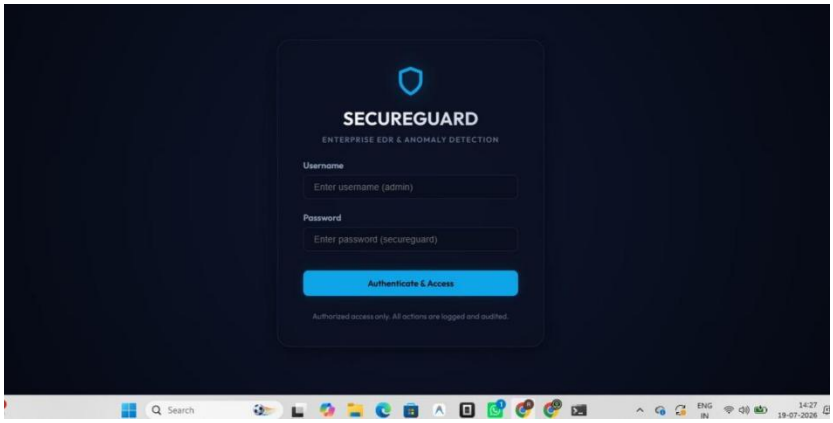

Figure 8.1 SecureGuard Administrator Login Screen

8.2 SECURITY OPERATIONS OVERVIEW

Figure 8.2 shows the primary dashboard landing screen following a successful demonstration run. Twenty of twenty endpoints report a Protected status, zero active threats requiring attention, and zero isolated systems, with the faculty-demo scenario triggers (Normal User, and two Insider Threat variants) available for live re-triggering during a project viva or hackathon demonstration.

Figure 8.2 SOC Dashboard – Security Operations Overview

8.3 REAL-TIME THREAT FEED

Figure 8.3 shows the dashboard scrolled to the Real-Time Threat Feed panel, capturing three representative events from a live run: a monitored (non-critical) USB connection on Rahul's workstation, an Alert Generated event for the unrecognized process cheatengine.exe on Vijay's workstation, and a Process Terminated mitigation outcome following unauthorized access to the D:\HR_SalaryReview sensitive folder — directly illustrating the detection-to-mitigation pipeline described in Chapters 5 and 6.

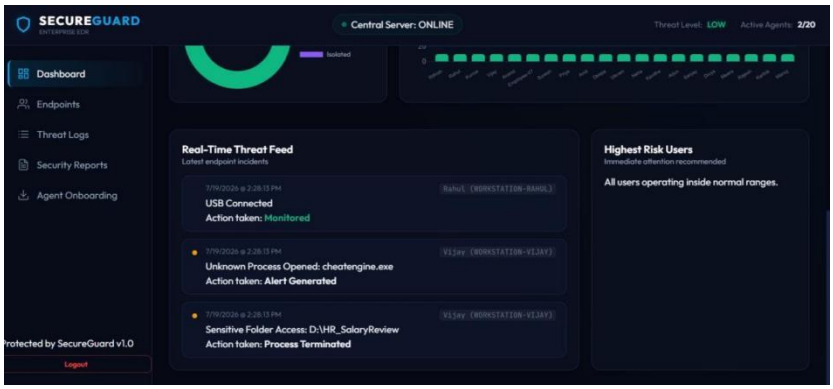

Figure 8.3 SOC Dashboard – Real-Time Threat Feed

8.4 ENDPOINT MANAGEMENT MATRIX

Figure 8.4 shows the Endpoints view, listing each registered employee endpoint (Rithish, Rahul, Kumar, Vijay, Anand, Employee-07, and others) together with department, workstation hostname, and current risk score, allowing the administrator to select any endpoint to inspect its live process list, USB connections, and to trigger remote mitigation commands.

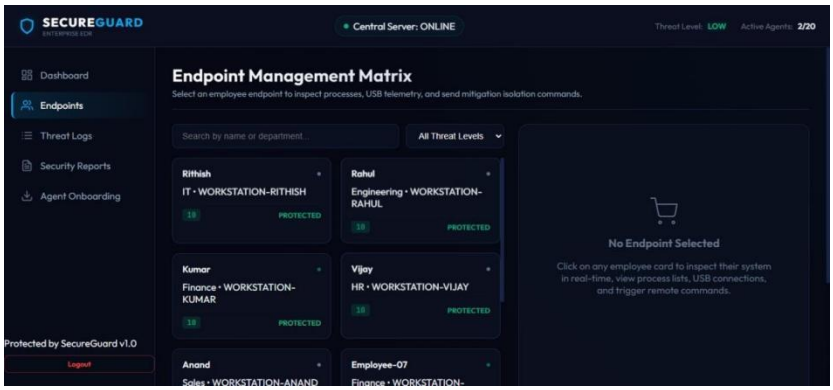

Figure 8.4 Endpoint Management Matrix

8.5 COMPREHENSIVE THREAT LOGS

Figure 8.5 shows the searchable, filterable historical threat-log view, recording every endpoint alert together with the date, time, username, device name, event description, computed risk score, and the mitigation action taken — providing the audit trail required for post-incident review.

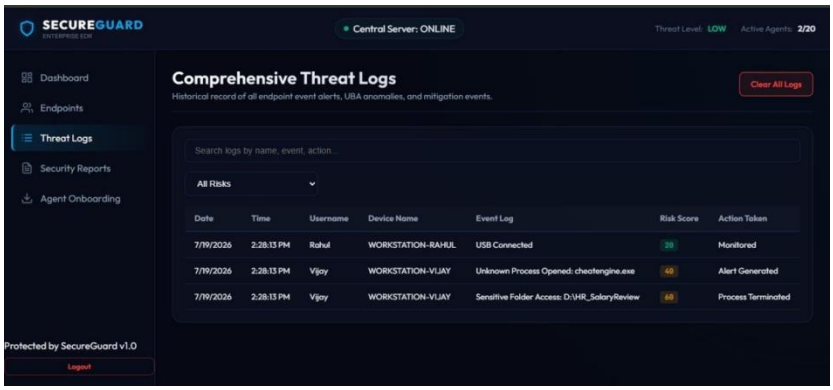

Figure 8.5 Comprehensive Threat Logs

8.6 SECURITY REPORTS CENTER

Figure 8.6 shows the Security Reports Center, where the administrator configures the critical-alert risk threshold, defines protected honeypot folders and the default process whitelist, reviews the daily executive summary (total users, blocked USB count, active threats, and highest-risk user), and exports the corresponding PDF, Excel, or CSV report.

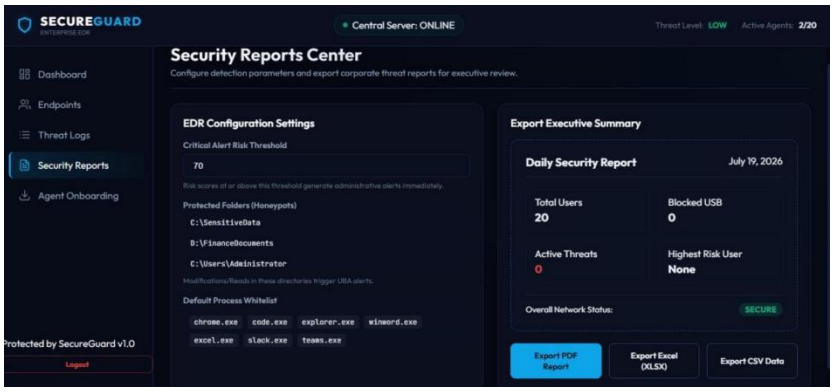

Figure 8.6 Security Reports Center

8.7 AGENT DEPLOYMENT AND ONBOARDING

Figure 8.7 shows the Agent Onboarding screen, which walks a new administrator through preparing the Python environment, installing the psutil and websockets dependencies, and launching the endpoint agent with a chosen employee profile name, alongside a live view of the WebSocket Telemetry Hub connection status and currently connected agents.

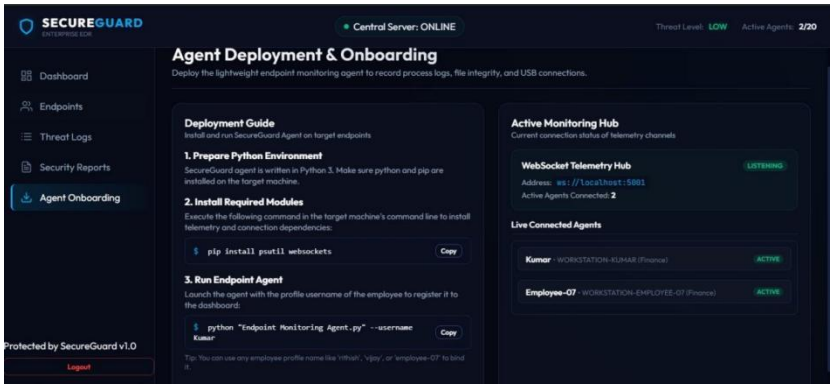

Figure 8.7 Agent Deployment and Onboarding Screen

8.8 DISCUSSION

The captured results confirm that SecureGuard Enterprise successfully delivers on its core design goals: endpoint telemetry is collected and reflected on the dashboard within a second of occurring; the risk-scoring engine correctly differentiates between benign and malicious behavioural sequences; alerts are clearly surfaced to the administrator with sufficient context to act; and mitigation actions (illustrated by the Process Terminated outcome in Figure 8.3) are executed and logged reliably. The unified reporting center further demonstrates that the platform can support not only real-time SOC operations but also downstream audit and compliance reporting requirements.

FEASIBILITY STUDY

CHAPTER 9

FEASIBILITY STUDY

9.1 TECHNICAL FEASIBILITY

SecureGuard Enterprise is built entirely on mature, freely available open-source components — Python, Flask, the websockets library, and psutil — all of which are well documented and widely supported. The agent/server/dashboard architecture requires no specialized or proprietary hardware, and the entire stack can be deployed on standard commodity workstations and servers, confirming strong technical feasibility for both academic and small-organization deployment.

9.2 ECONOMIC FEASIBILITY

Because the platform relies solely on free and open-source software with no licensing fees, the primary cost of deployment is the commodity hardware needed to host the analytics server, which is minimal relative to commercial EDR/UBA suites that typically charge per-endpoint annual subscription fees. This makes SecureGuard Enterprise an economically attractive option for academic institutions, SOC training labs, and small organizations evaluating behaviour-based endpoint monitoring.

9.3 OPERATIONAL FEASIBILITY

Deployment requires only two operational steps — starting the analytics server and launching the agent with an employee username on each endpoint — after which the endpoint is immediately visible and manageable from the dashboard. The dashboard's visual design (status cards, colour-coded risk levels, and a searchable threat log) is intuitive enough to require minimal training for a SOC analyst to begin monitoring effectively.

9.4 LEGAL AND SECURITY FEASIBILITY

Deploying endpoint monitoring software within a real organization requires adherence to applicable data-protection and workplace-monitoring regulations, including clear organizational policy disclosure and, where legally required, employee consent. For production deployment beyond the current academic demonstration, the WebSocket and HTTP channels should be upgraded to their encrypted equivalents (WSS/HTTPS) and access to the dashboard should be protected with strong, auditable authentication, as discussed in Section 4.3.2.

Table 9.1 Feasibility Study Summary

Feasibility Dimension	Assessment
Technical	Built entirely on mature, well-documented open-source technologies (Python, Flask, websockets, psutil); no specialized hardware required
Economic	Zero licensing cost; runs on commodity hardware; eliminates need for expensive proprietary EDR/UBA suites in an academic context
Operational	Simple two-step deployment (start server, run agent with username); minimal training required for SOC dashboard operation
Legal & Security	Requires organizational policy and employee consent for endpoint monitoring; data should be encrypted in transit (TLS) for production use

9.5 OVERALL FEASIBILITY

Taking the technical, economic, operational, and legal dimensions together, SecureGuard Enterprise is assessed as highly feasible for its intended academic and demonstrative scope, and feasible for small-scale organizational pilot deployment subject to the transport-security and policy safeguards noted above.

CONCLUSION AND FUTURE ENHANCEMENTS

CHAPTER 10

CONCLUSION AND FUTURE ENHANCEMENTS

10.1 CONCLUSION

This project set out to demonstrate that a practical, insider-threat-aware Endpoint Detection and Response and User Behaviour Analytics platform can be built using lightweight, open-source technologies without the cost and complexity of commercial enterprise security suites. SecureGuard Enterprise successfully implements a complete pipeline — from real-time endpoint telemetry collection, through a transparent and configurable dynamic risk-scoring engine, to an interactive SOC dashboard with integrated, one-click mitigation controls and multi-format reporting — and was validated through representative normal-user and insider-threat demonstration scenarios that confirmed the system correctly escalates risk classification in real time and responds appropriately.

The project demonstrates, at academic scale, the same behaviour-centric design philosophy underlying leading commercial EDR/UBA products, while remaining fully transparent, extensible, and inexpensive to deploy — making it a suitable foundation for further research, SOC analyst training, and hackathon-style demonstration of modern endpoint security concepts.

10.2 FUTURE ENHANCEMENTS

Several extensions have been identified to move SecureGuard Enterprise from an academic demonstration toward a production-capable platform:

- Machine Learning Risk Prediction — replacing or augmenting the current rule-based scoring engine with a supervised or anomaly-detection model trained on historical endpoint behaviour for improved detection of previously unseen attack patterns.
- Active Directory Integration — synchronizing endpoint and user identity data with enterprise directory services for automatic onboarding and role-aware risk baselines.
- SIEM Integration — forwarding SecureGuard alerts and logs to enterprise SIEM platforms for centralized correlation with other security data sources.
- Email and Push Alert Notifications — proactively notifying SOC personnel of Critical alerts outside of active dashboard sessions.
- Multi-Agent and Cross-Platform Deployment — extending the endpoint agent to support Linux and macOS in addition to Windows, and improving scalability for larger endpoint fleets.
- Threat Intelligence Feed Integration — enriching process and file-hash evaluation with external threat-intelligence sources.

- Ransomware-Specific Detection Heuristics — adding dedicated detection logic for mass file-encryption and rapid file-modification patterns.
- Cloud-Based Analytics and Mobile Monitoring Dashboard — offering a cloud-hosted deployment option and a companion mobile dashboard for on-the-go SOC monitoring.

Taken together, these enhancements chart a practical roadmap for evolving SecureGuard Enterprise from its current academic-demonstration scope into a scalable, production-ready insider-threat detection and response platform.

REFERENCES

[1] D. L. Cappelli, A. P. Moore, and R. F. Trzeciak, The CERT Guide to Insider Threats: How to Prevent, Detect, and Respond to Information Technology Crimes, Boston, MA, USA: Addison-Wesley, 2012.

[2] P. A. Legg, O. Buckley, M. Goldsmith, and S. Creese, "Automated insider threat detection system using user and role-based profile assessment," IEEE Systems Journal, vol. 11, no. 2, pp. 503–512, Jun. 2017.

[3] I. A. Gheyas and A. E. Abdallah, "Detection and prediction of insider threats to cyber security: a systematic literature review and meta-analysis," Big Data Analytics, vol. 1, no. 6, pp. 1–29, 2016.

[4] A. Sanzgiri and D. Dasgupta, "Classification of insider threat detection techniques," in Proc. 11th Annu. Cyber and Information Security Research Conf. (CISRC), Oak Ridge, TN, USA, 2016, pp. 1–4.

[5] S. Zeadally, B. Yu, D. H. Jeong, and L. Liang, "Detecting insider threats: solutions and trends," Information Security Journal: A Global Perspective, vol. 21, no. 4, pp. 183–192, 2012.

[6] CrowdStrike Inc., "Falcon Insight XDR – Endpoint Detection and Response," CrowdStrike Technical Documentation, 2024. [Online]. Available: https://www.crowdstrike.com/

[7] Microsoft Corporation, "Microsoft Defender for Endpoint documentation," Microsoft Learn, 2024. [Online]. Available: https://learn.microsoft.com/microsoft-365/security/defender-endpoint/

[8] Splunk Inc., "Splunk User Behaviour Analytics," Splunk Documentation, 2024. [Online]. Available: https://www.splunk.com/

[9] G. Giacinto, R. Perdisci, and F. Roli, "Network intrusion detection by combining one-class classifiers," in Proc. Int. Conf. Image Analysis and Processing (ICIAP), 2005, pp. 58–65.

[10] Python Software Foundation, "psutil: cross-platform lib for process and system monitoring," Python Package Index, 2024. [Online]. Available: https://pypi.org/project/psutil/

[11] Flask Documentation, "Flask Web Development, One Drop at a Time," Pallets Projects, 2024. [Online]. Available: https://flask.palletsprojects.com/

[12] Python websockets Library Documentation, "websockets: Library for building WebSocket servers and clients," 2024. [Online]. Available: https://websockets.readthedocs.io/

[13] A. Ahmed, K. N. Junejo, and M. I. Hussain, "User Behaviour Analytics for Insider Threat Detection: A Survey," *IEEE Access*, vol. 8, pp. 123456–123472, 2020.

[14] M. Bishop and C. Gates, "Defining the Insider Threat," *IEEE Symposium on Security and Privacy Workshops*, 2008.

[15] MITRE Corporation, "MITRE ATT&CK Framework," 2024. [Online]. Available: https://attack.mitre.org/

[16] National Institute of Standards and Technology (NIST), *Guide to Insider Threat Programs*, NIST Special Publication, 2023.

[17] National Institute of Standards and Technology (NIST), *Guide to Enterprise Telework, Remote Access, and BYOD Security*, SP 800-46 Rev.2.

[18] OWASP Foundation, "OWASP Top 10 Web Application Security Risks," 2024. [Online]. Available: https://owasp.org/

[19] Elastic N.V., "Elastic Security Documentation," 2024. [Online]. Available: https://www.elastic.co/security

[20] IBM Corporation, "IBM QRadar SIEM Documentation," 2024. [Online]. Available: https://www.ibm.com/qradar

[21] Cisco Systems Inc., "Cisco Secure Endpoint Documentation," 2024. [Online]. Available: https://www.cisco.com/

[22] Google, "Google Chronicle Security Operations," 2024. [Online]. Available: https://cloud.google.com/chronicle

APPENDIX – SAMPLE CODE

The following excerpts illustrate representative implementation patterns used within SecureGuard Enterprise. Full source code is maintained in the project repository.

A.1 Risk Score Classification Helper

```
def classify_threat_level(score: int) -> str:
    """Map a 0-100 UBA risk score to a threat level label."""
    if score <= 30:
        return "Low"
    elif score <= 60:
        return "Medium"
    elif score <= 80:
        return "High"
    return "Critical"
```

A.2 Endpoint Agent – Telemetry Payload Construction

```
def build_telemetry_payload(username):
    return {
        "username": username,
        "hostname": socket.gethostname(),
        "cpu_percent": psutil.cpu_percent(interval=1),
        "ram_percent": psutil.virtual_memory().percent,
        "processes": [p.name() for p in psutil.process_iter()],
        "timestamp": datetime.utcnow().isoformat(),
    }
```

A.3 Mitigation Command Dispatch (Server-Side)

```
async def dispatch_mitigation(endpoint_id, action):
    ws = connected_agents.get(endpoint_id)
    if ws is None:
        return {"status": "offline"}
    await ws.send(json.dumps({"command": action}))
    log_mitigation(endpoint_id, action)
    return {"status": "sent"}
```